STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory, Edge Cases & Code Examples

- ✓ The Golden Rule
- ✓ Container-Specific Thread Safety Characteristics
- ✓ Why operator[] is Dangerous on Maps
- ✓ Iterator Invalidation in Concurrent Contexts
- ✓ C++ Standard Guarantees (§23.2.2 since C++11)

→ *Swipe to learn* →

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 1. STL Thread Safety - The Golden Rule

Core Guarantee (C++ Standard §23.2.2):

Access Pattern	Thread Safety	Synchronization Required?
Multiple const reads	✅ Safe	❌ No (C++11 guarantee)
Multiple writes	❌ Data race	✅ Yes (mutex/lock)
Read + write	❌ Data race	✅ Yes (mutex/lock)
Writes to different containers	✅ Safe	❌ No

Simple Rule:

```
const std::vector<int> vec = {1, 2, 3};

// ✅ SAFE: Concurrent reads on const container
void thread1() { int x = vec[0]; }
void thread2() { int y = vec[1]; }

// ❌ UNSAFE: Concurrent writes without synchronization
std::vector<int> vec2;
void writer1() { vec2.push_back(1); }  // DATA RACE
void writer2() { vec2.push_back(2); }  // DATA RACE
```

Critical Insight: operator[] is a WRITE on Maps

```
std::map<int, string> map;

map[1];  // ❌ If key 1 doesn't exist, INSERTS default value!
         // This is a WRITE operation, not a read

// ✅ Use at() or find() for read-only access:
map.at(1);        // Throws if key missing
map.find(1);      // Returns end() if missing
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 2. Container-Specific Thread Safety Characteristics (1/2)

Internal Implementation Differences:

Container	Internal Structure	Reallocation Trigger	Iterator Invalidation Scope	Thread Safety Risk
std::vector	Dynamic array	`push_back()`, `insert()`, `resize()` when capacity exceeded	❌ ALL iterators, refs, pointers	HIGH - global reallocation
std::map	Red-black tree	Never (grows node-by-node)	✅ Only erased elements (single-thread)	MEDIUM - localized rebalancing
std::unordered_map	Hash table	Insert when load factor > threshold	❌ ALL iterators when rehashing	VERY HIGH - global rehashing
std::list	Doubly-linked list	Never (grows node-by-node)	✅ Only erased elements	LOW - stable iterators (still needs sync)
std::deque	Array of arrays	Insert in middle	❌ ALL if middle insert	MEDIUM-HIGH

Detailed Characteristics:

std::vector:

```
std::vector<int> vec;
vec.reserve(10);  // Capacity = 10

// Thread 1: Reading
int x = vec[0];

// Thread 2: May trigger reallocation
vec.push_back(11);  // If size >= capacity → reallocate entire buffer
                    // Thread 1's access becomes dangling pointer → CRASH
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 2. Container-Specific Thread Safety Characteristics (2/2)

std::map (Red-Black Tree):

```
std::map<int, string> tree_map;

// Insert only modifies nodes along path from root to insertion point
tree_map[50] = "data";  // Rebalances only 3-4 nodes (O(log n))
                        // Other nodes untouched
// Still UNSAFE for concurrent access - tree structure modified
```

std::unordered_map (Hash Table):

```
std::unordered_map<int, string> hash_map;
// Default: max_load_factor = 1.0

hash_map[1] = "a";  // OK
// ... insert 99 more elements ...
hash_map[100] = "z";  // May trigger rehashing:
                       // 1. Allocate new bucket array (2x size)
                       // 2. Rehash ALL 100 elements
                       // 3. Invalidate ALL iterators
                       // 4. Deallocate old array
```

Solution - Always reserve() for hash tables:

```
hash_map.reserve(200);  // Pre-allocate to prevent rehashing
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 3. Why operator[] is Dangerous on Maps (1/2)

The Hidden Write Problem:

`operator[]` is **NOT a read operation** - it always has potential to write!

What operator[] Actually Does:

Step	Operation	Type	Risk
1	Search for key	Read	Safe if const
2	If key exists, return reference	Read	Safe if const
3	If key missing, **insert default value**	❌ Write	Modifies container!
4	If insert exceeds load factor, **rehash**	❌ Global write	Invalidates ALL iterators!

Example - Surprising Writes:

```
std::unordered_map<int, std::string> map;

// Thread 1
map[1] = "hello";  // If key 1 doesn't exist:
                   // 1. Insert {1, ""}  (default-constructed string)
                   // 2. Assign "hello"
                   // This is a WRITE, not a read!

// Thread 2
auto val = map[2];  // If key 2 doesn't exist:
                    // 1. Insert {2, ""}
                    // 2. Return reference to ""
                    // May trigger REHASHING if load factor exceeded
                    // ❌ DATA RACE with Thread 1
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 3. Why operator[] is Dangerous on Maps (2/2)

Safe Alternatives:

Method	Const-Safe?	Throws on Missing?	Returns on Missing	Use Case
`operator[]`	❌ No	No (inserts)	Default value	Write/insert operations
`at(key)`	✅ Yes	Yes (`std::out_of_range`)	N/A	Read-only with exception
`find(key)`	✅ Yes	No	`end()` iterator	Read-only without exception

Correct Read-Only Access:

```
const std::map<int, string> config = {{1, "one"}, {2, "two"}};

// ❌ COMPILATION ERROR - operator[] is non-const
// auto val = config[1];

// ✅ Use at() - throws if missing
try {
    auto val = config.at(1);  // Returns "one"
} catch (const std::out_of_range& e) {
    // Handle missing key
}

// ✅ Use find() - returns end() if missing
auto it = config.find(1);
if (it != config.end()) {
    auto val = it->second;  // Returns "one"
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 4. Iterator Invalidation in Concurrent Contexts (1/2)

Critical Difference from Single-Threaded Code:

In single-threaded code, you control when invalidation happens. In multithreaded code, **any thread can invalidate iterators at any time** without synchronization.

The Time-of-Check to Time-of-Use (TOCTOU) Race:

Timeline	Thread 1	Thread 2	Iterator State
T0	`auto it = vec.begin()`	-	✅ Valid (points to buffer A)
T1	... processing ...	`vec.push_back(4)`	❌ Invalid (buffer A freed)
T2	`int val = *it`	-	CRASH - dangling pointer!

Example - Vector Reallocation Race:

```
std::vector<int> vec = {1, 2, 3};  // capacity = 3

// Thread 1: Iterator user
auto it = vec.begin();  // Points to buffer at address 0x1000
std::this_thread::sleep_for(10ms);  // Processing delay
int val = *it;  // ❌ May access freed memory!

// Thread 2: Modifier
vec.push_back(4);  // Reallocates to new buffer at 0x2000
                   // Old buffer (0x1000) freed
                   // Thread 1's iterator now dangling
```

Why No Happens-Before Relationship:

Without synchronization primitives (mutex, atomic, etc.), there's **no happens-before edge** between:
- Thread 1 obtaining iterator
- Thread 2 modifying container

Even if T1's `vec.begin()` executes "first" in wall-clock time, the C++ memory model doesn't guarantee T1 will observe the container in a consistent state.

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 4. Iterator Invalidation in Concurrent Contexts (2/2)

Container-Specific Invalidation in Concurrent Context:

Container	Single-Thread Guarantee	Multi-Thread Reality
`vector`	Insert/erase invalidates after point	❌ ANY modification invalidates ALL iterators across threads
`map`	Insert doesn't invalidate existing	❌ Tree rebalancing causes data race on structure
`unordered_map`	Insert invalidates if rehash	❌ Rehashing invalidates ALL + data race on bucket array
`list`	Insert doesn't invalidate others	❌ Concurrent insert causes data race on next/prev pointers

Key Insight:

Single-threaded invalidation rules describe **which iterators become invalid**. Multithreaded reality is **all concurrent access without sync is undefined behavior**, regardless of invalidation rules.

Safe Pattern - Lock During Iterator Lifetime:

```
std::vector<int> vec;
std::mutex mtx;

// Thread 1: Safe iterator usage
{
    std::lock_guard lock(mtx);
    auto it = vec.begin();
    // Iterator valid while lock held
    int val = *it;
}  // Lock released

// Thread 2: Safe modification
{
    std::lock_guard lock(mtx);
    vec.push_back(4);  // Exclusive access
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 5. C++ Standard Guarantees (§23.2.2 since C++11) (1/2)

Formal Thread Safety Rules:

Access Pattern	Example	Synchronization Required?	Standard Guarantee
Multiple reads on const container	`const vec`: T1 reads `[0]`, T2 reads `[1]`	❌ No	✅ Safe (C++11+)
Writes to different containers	T1 writes `vec1`, T2 writes `vec2`	❌ No	✅ Safe (different objects)
Multiple writes to same container	T1 `push_back`, T2 `push_back`	✅ Yes (mutex)	❌ Data race without sync
Read + write to same container	T1 reads `[0]`, T2 `push_back`	✅ Yes (mutex)	❌ Data race without sync
Non-const read + non-const read	`vec` (non-const): T1 reads, T2 reads	⚠️ Unclear	❌ NOT guaranteed safe

Critical Distinction: Const vs Non-Const Access:

```
// ✅ SAFE - const-qualified reads
const std::vector<int> vec1 = {1, 2, 3};
void thread1() { int x = vec1[0]; }  // Safe
void thread2() { int y = vec1[1]; }  // Safe

// ❌ NOT GUARANTEED SAFE - non-const reads
std::vector<int> vec2 = {1, 2, 3};  // Non-const
void thread3() { int x = vec2[0]; }  // Not guaranteed safe!
void thread4() { int y = vec2[1]; }  // Not guaranteed safe!
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Theory

📌 5. C++ Standard Guarantees (§23.2.2 since C++11) (2/2)

Why Non-Const Reads Aren't Guaranteed Safe:

Historical Reason (Pre-C++11):

- Copy-on-write strings modified reference counts even during reads
- Non-const access could modify internal state (lazy initialization, caching, etc.)

Modern Reason (C++11+):

- Debug builds may track iterator states in non-const containers
- Implementation may use internal mutexes for non-const access
- No guarantee that "logically read-only" operations don't modify state

The 4 Standard Guarantees in Detail:

1. **Const-Qualified Concurrent Reads: ✅ Safe**
   `cpp const std::map<int, string> config = {{1, "one"}}; // Thread 1, 2, 3 all read concurrently - guaranteed safe`
2. **Different Containers: ✅ Safe**
   `cpp std::vector<int> vec1, vec2; // Thread 1 modifies vec1, Thread 2 modifies vec2 - safe`
3. **Same Container Writes: ❌ Requires Sync**
   `cpp std::vector<int> vec; std::mutex mtx; // Thread 1, 2 must lock mtx before push_back`
4. **Same Container Read+Write: ❌ Requires Sync**
   `cpp std::vector<int> vec; std::mutex mtx; // Thread 1 (reader) and Thread 2 (writer) must both lock`

Key Takeaway:

The standard ONLY guarantees thread safety for **const-qualified container access**. Everything else requires explicit synchronization, even seemingly read-only operations on non-const containers.

STL Container Thread Safety - Concurrent Access Rules and Patterns

Edge Cases

📌 1. operator[] Triggers Rehashing

```
std::unordered_map<int, std::string> sensor_data;
sensor_data.reserve(100);  // Pre-allocate for 100 elements

// Thread 1: Camera sensor
sensor_data[1] = "camera_front";

// Thread 2: Lidar sensor
sensor_data[2] = "lidar_top";

// ... 98 more insertions ...

// Thread 100: Radar sensor
sensor_data[100] = "radar_left";  // ✅ Still safe - within reserved capacity

// Thread 101: GPS sensor
sensor_data[101] = "gps";  // ❌ TRIGGERS REHASHING! Invalidates all iterators across all threads
```

Why this happens:

- `reserve(100)` allocates buckets for ~100 elements based on load factor
- Insertion 101 exceeds capacity
- Hash table rehashes, moving all 100 existing elements
- Any thread holding an iterator or pointer to an element now has dangling reference

Solution:

```
sensor_data.reserve(200);  // Over-allocate to prevent rehashing
// OR
std::shared_mutex mtx;
{
    std::unique_lock lock(mtx);  // Exclusive lock for writes
    sensor_data[101] = "gps";
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Edge Cases

📌 2. std::map::operator[] is NOT Const

```
const std::map<int, std::string> const_map = {{1, "one"}, {2, "two"}};

// ❌ COMPILATION ERROR
// auto val = const_map[1];  // operator[] is non-const!

// ✅ Correct - use at() for const access
auto val = const_map.at(1);  // Throws if key doesn't exist

// ✅ Or use find()
auto it = const_map.find(1);
if (it != const_map.end()) {
    auto val = it->second;
}
```

Implication for multithreading:
Even if you only want to read from a map, you cannot use `operator[]` on a `const std::map&`. This is by design: `operator[]` **modifies** the map if the key is missing. Use `at()` (throws exception) or `find()` (returns iterator) for read-only access.

STL Container Thread Safety - Concurrent Access Rules and Patterns

Edge Cases

📌 3. Vector Bool is NOT Thread-Safe Even for Separate Elements

```
std::vector<bool> flags(1000);  // Specialized for space efficiency

// Thread 1
flags[0] = true;

// Thread 2
flags[1] = true;  // ❌ DATA RACE even though indices differ!
```

Why:

`std::vector<bool>` is a **space-optimized specialization** that packs bools into bits. Multiple "elements" share the same underlying byte. Modifying `flags[0]` and `flags[1]` may modify the same byte, causing a data race at the hardware level (read-modify-write on the same memory location).

Solution:

```
std::vector<char> flags(1000);  // Each element is separate byte - no false sharing
// OR
std::vector<std::atomic<bool>> flags(1000);  // Atomic elements (if lock-free)
// OR
Use external synchronization (mutex per group of bits)
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Edge Cases

📌 4. Iterators and References During Concurrent Modification

```
std::map<int, std::string> config;
config[1] = "setting1";
config[2] = "setting2";

// Thread 1: Iterator loop
for (auto& pair : config) {
    std::cout << pair.second << "\n";  // Reading
}

// Thread 2: Concurrent insertion
config[3] = "setting3";  // ❌ Modifies container during iteration
```

Even though `std::map` insertion doesn't invalidate iterators to existing elements (in single-threaded code), **concurrent modification during iteration is a data race**. The internal tree structure is being modified while Thread 1 traverses it, leading to undefined behavior.

Failure modes:
- Crash due to inconsistent tree pointers
- Infinite loop if tree structure becomes cyclic during rebalancing
- Skipped or duplicate elements in iteration
- Silent data corruption

STL Container Thread Safety - Concurrent Access Rules and Patterns

Edge Cases

📌 5. std::string Copy-on-Write (Pre-C++11)

```
// Pre-C++11 implementations (e.g., old libstdc++)
std::string shared = "shared_data";

// Thread 1
char c = shared[0];  // Read operation

// Thread 2
char d = shared[1];  // Another read

// Internal COW (copy-on-write) implementation:
// - Both threads access reference count
// - Reference count increment/decrement = data race!
```

Historical context:

Pre-C++11 `std::string` implementations (like GCC 4.x libstdc++) used copy-on-write with reference counting. Even **read operations** modified internal reference counts, causing data races. C++11 **banned COW strings** to enable thread-safe const access.

Modern guarantee (C++11+):

const `std::string` access from multiple threads is guaranteed safe. All major implementations (libc++, libstdc++, MSVC) use **eager copying** or **small string optimization (SSO)**, not COW.

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 1. Safe Concurrent Reads (Const Qualification) (1/2)

Key points:

- Container is `const` qualified
- Multiple threads read simultaneously
- No writes, no data races
- Standard guarantees thread safety for this pattern

```
#include <iostream>
#include <vector>
#include <thread>
#include <string>

// ✅ SAFE: Concurrent reads on const-qualified container
void safe_concurrent_reads() {
    const std::vector<std::string> sensor_names = {
        "camera_front", "lidar_top", "radar_left", "gps"
    };

    auto reader = [&](int id) {
        // Multiple threads reading different elements - SAFE
        for (int i = 0; i < 1000; ++i) {
            std::string name = sensor_names[id % sensor_names.size()];
            // Process sensor name...
        }
    };

    std::thread t1(reader, 0);
    std::thread t2(reader, 1);
    std::thread t3(reader, 2);

    t1.join();
    t2.join();
    t3.join();

    std::cout << "Concurrent reads completed safely\n";
}

int main() {
    safe_concurrent_reads();
// ...continues on next slide
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 1. Safe Concurrent Reads (Const Qualification) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    return 0;
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 2. Unsafe Concurrent Writes (Data Race)

Failure modes:

1. **Lost writes**: Both threads read same size, increment, write back → one write lost
2. **Reallocation race**: T1 starts realloc, T2 writes to old buffer → write lost or corruption
3. **Crash**: T1 frees old buffer, T2 still writing to it → segfault

---

```
#include <iostream>
#include <vector>
#include <thread>

// ❌ UNSAFE: Concurrent writes cause data race
void unsafe_concurrent_writes() {
    std::vector<int> sensor_readings;  // Non-const, shared

    auto writer = [&](int id) {
        for (int i = 0; i < 100; ++i) {
            sensor_readings.push_back(id * 1000 + i);  // ❌ DATA RACE!
        }
    };

    std::thread t1(writer, 1);
    std::thread t2(writer, 2);

    t1.join();
    t2.join();

    // Undefined behavior occurred during concurrent push_back
    // Possible outcomes:
    // - Size is less than 200 (lost writes)
    // - Crash during reallocation
    // - Memory corruption
    std::cout << "Vector size: " << sensor_readings.size() << " (expected 200)\n";
}

int main() {
    unsafe_concurrent_writes();  // ⚠️ Will likely crash or give wrong results
    return 0;
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 3. Safe Concurrent Writes with Mutex

Performance consideration:

Locking on every `push_back` is expensive. Better approach: each thread writes to local vector, then merge:

```
#include <iostream>
#include <vector>
#include <thread>
#include <mutex>

// ✅ SAFE: Mutex protects concurrent writes
void safe_concurrent_writes_with_mutex() {
    std::vector<int> sensor_readings;
    std::mutex mtx;

    auto writer = [&](int id) {
        for (int i = 0; i < 100; ++i) {
            std::lock_guard<std::mutex> lock(mtx);  // ✅ Exclusive access
            sensor_readings.push_back(id * 1000 + i);
        }
    };

    std::thread t1(writer, 1);
    std::thread t2(writer, 2);

    t1.join();
    t2.join();

    std::cout << "Vector size: " << sensor_readings.size() << " (expected 200)\n";
    // Always prints 200 - no data race
}

int main() {
    safe_concurrent_writes_with_mutex();
    return 0;
}
```

```
['void optimized_concurrent_writes() {\n    std::vector<int> sensor_readings;\n    std::mutex mtx;\n\n    auto writer =
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 4. Reader-Writer Lock for Read-Heavy Workloads (1/2)

When to use shared_mutex:

- Read-heavy workloads (10:1 or higher read:write ratio)
- Readers acquire shared lock (multiple allowed simultaneously)
- Writers acquire unique lock (exclusive access)
- Autonomous driving: Sensor configuration database (frequent reads, rare updates)

```
#include <iostream>
#include <map>
#include <thread>
#include <shared_mutex>
#include <vector>
#include <chrono>

using namespace std::chrono_literals;

// ✅ SAFE: shared_mutex allows multiple readers, exclusive writer
class SensorDatabase {
private:
    std::map<int, std::string> sensors;
    mutable std::shared_mutex mtx;  // mutable for const member functions

public:
    // Writer: Exclusive lock
    void add_sensor(int id, const std::string& name) {
        std::unique_lock lock(mtx);  // Exclusive lock
        sensors[id] = name;
        std::cout << "Added sensor: " << id << "\n";
    }

    // Reader: Shared lock (multiple readers allowed)
    std::string get_sensor(int id) const {
        std::shared_lock lock(mtx);  // Shared lock
        auto it = sensors.find(id);
        return (it != sensors.end()) ? it->second : "NOT_FOUND";
    }

    // Reader: Shared lock
    size_t count() const {
// ...continues on next slide
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 4. Reader-Writer Lock for Read-Heavy Workloads (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        std::shared_lock lock(mtx);
        return sensors.size();
    }
};

void demonstrate_reader_writer() {
    SensorDatabase db;

    // Writer thread: Adds sensors periodically
    std::thread writer([&]() {
        for (int i = 1; i <= 5; ++i) {
            db.add_sensor(i, "Sensor_" + std::to_string(i));
            std::this_thread::sleep_for(100ms);
        }
    });

    // Multiple reader threads: Query concurrently
    auto reader = [&](int id) {
        for (int i = 0; i < 20; ++i) {
            std::string name = db.get_sensor(id);
            // Multiple readers can execute simultaneously
            std::this_thread::sleep_for(10ms);
        }
    };

    std::vector<std::thread> readers;
    for (int i = 0; i < 5; ++i) {
        readers.emplace_back(reader, i % 5 + 1);
    }

    writer.join();
    for (auto& t : readers) t.join();

    std::cout << "Final count: " << db.count() << "\n";
}

int main() {
    demonstrate_reader_writer();
    return 0;
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 5. std::unordered_map Rehashing Pitfall (1/2)

Key lesson:
- **Always** call `reserve()` on `unordered_map` before concurrent access
- Calculate expected size upfront
- Over-allocate to be safe (e.g., reserve 150% of expected size)

```
#include <iostream>
#include <unordered_map>
#include <thread>
#include <vector>
#include <string>

// ❌ DANGEROUS: Rehashing invalidates iterators across threads
void demonstrate_rehashing_danger() {
    std::unordered_map<int, std::string> map;
    // No reserve() - rehashing will occur!

    // Thread 1: Inserts elements (may trigger rehashing)
    std::thread writer([&]() {
        for (int i = 0; i < 100; ++i) {
            map[i] = "data_" + std::to_string(i);
            // Rehashing may occur when load factor exceeds 1.0
        }
    });

    // Thread 2: Iterates over map
    std::thread reader([&]() {
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
        for (const auto& pair : map) {
            // ❌ If writer triggers rehashing during iteration:
            // - Iterator invalidated
            // - Undefined behavior (crash, infinite loop, corruption)
            std::cout << pair.first << ": " << pair.second << "\n";
        }
    });

    writer.join();
    reader.join();
}
// ...continues on next slide
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 5. std::unordered_map Rehashing Pitfall (2/2)

[Code continues across slides]

```
// ...continued from previous slide

// ✅ SAFE: Reserve prevents rehashing
void demonstrate_rehashing_prevention() {
    std::unordered_map<int, std::string> map;
    map.reserve(200);  // Pre-allocate for 200 elements

    std::mutex mtx;

    auto writer = [&](int start) {
        for (int i = start; i < start + 50; ++i) {
            std::lock_guard lock(mtx);
            map[i] = "data_" + std::to_string(i);
            // No rehashing - capacity pre-allocated
        }
    };

    std::thread t1(writer, 0);
    std::thread t2(writer, 50);

    t1.join();
    t2.join();

    std::cout << "Map size: " << map.size() << ", bucket count: "
              << map.bucket_count() << "\n";
}

int main() {
    // demonstrate_rehashing_danger();  // ⚠️ Likely crashes
    demonstrate_rehashing_prevention();  // ✅ Safe
    return 0;
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 6. Comparing std::map vs std::unordered_map Thread Safety (1/2)

Analysis:

- `std::map`: O(log n) insertion, but localized tree rebalancing
- `std::unordered_map`: O(1) average insertion, but global rehashing risk
- For thread safety: **both require synchronization**, but unordered_map needs `reserve()`

```
#include <iostream>
#include <map>
#include <unordered_map>
#include <thread>
#include <mutex>
#include <chrono>

using namespace std::chrono_literals;

// Demonstrate why unordered_map is more dangerous
void compare_map_vs_unordered_map() {
    std::map<int, int> tree_map;
    std::unordered_map<int, int> hash_map;
    hash_map.reserve(1000);  // Still need to reserve!

    std::mutex tree_mtx, hash_mtx;

    // Insert into std::map
    auto insert_map = [&]() {
        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < 1000; ++i) {
            std::lock_guard lock(tree_mtx);
            tree_map[i] = i * 2;
        }
        auto end = std::chrono::high_resolution_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
        std::cout << "map insertion: " << duration.count() << " µs\n";
    };

    // Insert into std::unordered_map
    auto insert_unordered = [&]() {
        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < 1000; ++i) {
// ...continues on next slide
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 6. Comparing std::map vs std::unordered_map Thread Safety (2/2)

[Code continues across slides]

```
// ...continued from previous slide
            std::lock_guard lock(hash_mtx);
            hash_map[i] = i * 2;
        }
        auto end = std::chrono::high_resolution_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);
        std::cout << "unordered_map insertion: " << duration.count() << " µs\n";
    };

    std::thread t1(insert_map);
    std::thread t2(insert_unordered);

    t1.join();
    t2.join();

    std::cout << "\n✅ Both completed safely with proper synchronization\n";
    std::cout << "map size: " << tree_map.size() << "\n";
    std::cout << "unordered_map size: " << hash_map.size() << "\n";
}

int main() {
    compare_map_vs_unordered_map();
    return 0;
}
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 7. Thread-Safe Wrapper Class (Production Pattern) (1/3)

Design patterns:

- RAII-style locking (automatic unlock)
- Reader-writer lock for read-heavy workloads
- `std::optional` for find (avoids throwing exceptions)
- Template for reusability
- Reserve in constructor to prevent rehashing

```
#include <iostream>
#include <unordered_map>
#include <shared_mutex>
#include <optional>
#include <string>

// ✅ Production-quality thread-safe map wrapper
template<typename Key, typename Value>
class ThreadSafeMap {
private:
    std::unordered_map<Key, Value> map;
    mutable std::shared_mutex mtx;

public:
    // Constructor with capacity hint
    explicit ThreadSafeMap(size_t reserve_size = 100) {
        map.reserve(reserve_size);
    }

    // Thread-safe insert
    void insert(const Key& key, const Value& value) {
        std::unique_lock lock(mtx);
        map[key] = value;
    }

    // Thread-safe find (returns optional)
    std::optional<Value> find(const Key& key) const {
        std::shared_lock lock(mtx);
        auto it = map.find(key);
        if (it != map.end()) {
// ...continues on next slide
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 7. Thread-Safe Wrapper Class (Production Pattern) (2/3)

```
// ...continued from previous slide
            return it->second;
        }
        return std::nullopt;
    }

    // Thread-safe contains
    bool contains(const Key& key) const {
        std::shared_lock lock(mtx);
        return map.find(key) != map.end();
    }

    // Thread-safe size
    size_t size() const {
        std::shared_lock lock(mtx);
        return map.size();
    }

    // Thread-safe erase
    bool erase(const Key& key) {
        std::unique_lock lock(mtx);
        return map.erase(key) > 0;
    }

    // Thread-safe clear
    void clear() {
        std::unique_lock lock(mtx);
        map.clear();
    }

    // Apply function to all elements (reader)
    template<typename Func>
    void for_each(Func func) const {
        std::shared_lock lock(mtx);
        for (const auto& pair : map) {
            func(pair.first, pair.second);
        }
    }
};

// Example: Sensor name registry for autonomous vehicle
// ...continues on next slide
```

STL Container Thread Safety - Concurrent Access Rules and Patterns

Code Examples

📌 7. Thread-Safe Wrapper Class (Production Pattern) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
void autonomous_vehicle_sensor_registry() {
    ThreadSafeMap<int, std::string> sensor_registry(50);

    // Writer thread: Registers sensors
    std::thread registration([&]() {
        sensor_registry.insert(1, "Camera_Front");
        sensor_registry.insert(2, "LiDAR_Top");
        sensor_registry.insert(3, "Radar_Front_Left");
        sensor_registry.insert(4, "GPS");
        sensor_registry.insert(5, "IMU");
        std::cout << "Registered 5 sensors\n";
    });

    // Reader thread: Queries sensors
    std::thread query([&]() {
        std::this_thread::sleep_for(std::chrono::milliseconds(10));

        auto camera = sensor_registry.find(1);
        if (camera) {
            std::cout << "Found sensor 1: " << *camera << "\n";
        }

        std::cout << "Total sensors: " << sensor_registry.size() << "\n";

        sensor_registry.for_each([](int id, const std::string& name) {
            std::cout << "  ID " << id << ": " << name << "\n";
        });
    });

    registration.join();
    query.join();
}

int main() {
    autonomous_vehicle_sensor_registry();
    return 0;
}
```

What You Learned

Key Takeaways

- ✓ The Golden Rule
- ✓ Container-Specific Thread Safety Characteristics
- ✓ Why operator[] is Dangerous on Maps
- ✓ Iterator Invalidation in Concurrent Contexts

→ *Swipe to learn* →